

Retrieval-Augmented Generation-Based Systems vs. Standalone Large Language Model Systems: A Comparative Analysis of System Trade-offs in Data-Driven Applications

Lok Yee Poh
Faculty of Computing
Universiti Teknologi Malaysia
Skudai, Johor Bahru
poh.yee@graduate.utm.my

Abstract— The increasing use of artificial intelligence in data-driven systems has led to the widespread adoption of Large Language Models (LLMs) for natural language processing tasks. However, standalone LLM systems are limited by their reliance on static trained data, which restricts their ability to access up-to-date or domain-specific knowledge during inference. Retrieval-Augmented Generation (RAG)-based systems address this limitation by integrating external knowledge retrieval into the generation process.

This paper presents a comparative study of standalone LLM systems and RAG-based systems, focusing on their architectural differences and performance across key dimensions, including accuracy, latency, cost, system complexity, and adaptability. A conceptual analysis approach is adopted to evaluate system behaviour across different operational scenarios.

The findings highlight clear trade-offs between both approaches, providing a structured understanding of their operational strengths and limitations. Standalone LLM systems are found to be more suitable for low-latency and simplified deployment environments, while RAG-based systems are more effective for knowledge-intensive applications requiring factual accuracy and external knowledge grounding. These findings provide practical guidance for system selection in real-world deployment, enabling developers and system architects to align architectural choices with performance requirements and operational constraints.

Keywords—Retrieval-Augmented Generation, Standalone Large Language Models, Data-Driven Decision System, Artificial Intelligence

I. INTRODUCTION

The increasing adoption of artificial intelligence and data-driven systems has greatly changed the way organizations process and utilise information. Large Language Model (LLM) have emerged as essential tools for natural language understanding and generation, supporting modern computing tasks such as conversational agents, automated summarization, and decision-support systems [1]. However, the utility of standalone LLM systems is inherently limited by their reliance on static training datasets. Regardless of their internal reasoning power, these models are restricted to the information available at their last training cutoff [2]. This constraint prevents them from accessing real-time or domain-specific data during inference, creating a critical need for new system architectures that can integrate external and dynamically updated knowledge sources.

To address this challenge, the Retrieval-Augmented Generation (RAG)-based systems were proposed as an alternative architectural design [3]. These systems combine the external retrieval system with language models to give them extra contextual information in response generation [1][3]. However, both standalone LLM systems and RAG-based systems present different operational trade-offs.

This paper aims to provide a comparative study of standalone LLM systems and RAG-based systems. The analysis focuses on key system dimensions such as accuracy, latency, cost, and architectural complexity. This comparison is intended to support practical decision-making in system design by helping practitioners understand how architectural choices influence performance under different operational requirements and constraints in real-world AI deployment.

II. SYSTEM ARCHITECTURE AND THEORETICAL BACKGROUND

A. Standalone Large Language Models System

Standalone LLMs are transformer-based neural network systems designed to generate natural language outputs by predicting the next token based on patterns learned during large-scale pretraining [4]. These models operate without requiring external data sources during inference, as all knowledge is encoded within the model parameters [1].

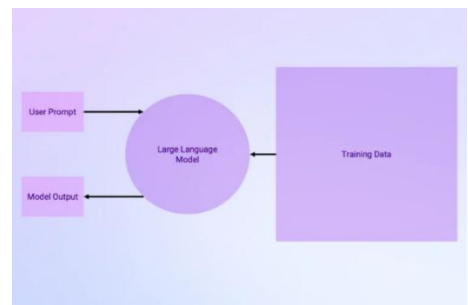


Fig. 1. Architecture of Standalone LLM

From an architectural perspective, standalone LLM follows a straightforward input–process–output flow as illustrated in Figure 1. In this monolithic architecture, the system relies entirely on a single-model pipeline where the Large Language Model acts as the central processing engine. When a User Prompt is received, the model processes the query by consulting its internal parameters—weights learned during the training phase using large-scale Training Data. Unlike integrated systems that fetch live data, this standalone design performs all computations within the model’s internal

boundary. The final Model Output is generated based on the statistical patterns learned from the training corpus. This design enables fast response generation and simplified deployment across general-purpose tasks such as text generation, summarization, translation, and dialogue systems [4].

A widely adopted real-world implementation of this architecture includes models such as GPT-3, which are deployed in applications such as conversational agents, content generation tools, and programming assistants [5]. These systems demonstrate strong generalization capability across diverse tasks but remain constrained by their inability to access external knowledge during inference.

B. Retrieval-Augmented Generation Based Models System

RAG-based systems are designed to improve the performance of language models by combining generative capabilities with external information retrieval [3]. Unlike standalone LLMs, RAG-based systems introduce an additional retrieval step before response generation.

The architecture of RAG-based systems consists of three main components: a retrieval module, an augmentation process, and a generative language model [2].

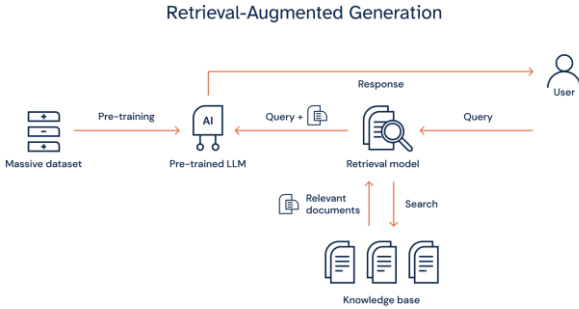


Fig. 2. Architecture of Retrieval-Augmented Generation (RAG) System

As illustrated in Figure 2, the process begins with the user's query, which triggers a parallel path where the retrieval module searches external data sources such as document databases or knowledge repositories (Knowledge Base) to obtain relevant documents, while the pre-trained LLM provides the foundational generation capability. These retrieved documents are then fed into the LLM alongside the original query and combined to form an enhanced input. The model processes this enriched input to generate a final response based on both the query and the retrieved context.

The key design principle of RAG-based systems is external knowledge augmentation, which allows models to dynamically extend their knowledge beyond the training corpus [3]. This architecture improves adaptability in rapidly changing or domain-specific environments, as new information can be incorporated without retraining the model.

The practical real-world implementation of RAG-based systems can be observed in Microsoft Bing Chat (Copilot) and Perplexity AI, which enhance large language model outputs by integrating web-scale retrieval mechanisms to produce up-to-date and source-grounded responses [6][7]. These systems illustrate how external knowledge integration can improve the

factual reliability and contextual relevance of output generated in real-world applications.

III. COMPARATIVE METHODOLOGY AND ANALYSIS

A. Approach

This study adopts a conceptual comparative approach without experimental implementation. Instead, the focus is on analysing how Standalone LLM systems and RAG-based systems operate and how their design differences influence performance in different scenarios. The analysis emphasises structural differences and practical behaviour in terms of accuracy, speed, cost, and system design.

B. Comparative Analysis

The comparison between Standalone LLM systems and RAG-based systems is summarized in Table I below, followed by a discussion of each criteria.

TABLE I. COMPARISON OF STANDALONE LLM AND RAG-BASED SYSTEMS

Criteria	Standalone LLM System	RAG-Based System
Accuracy	Strong general performance; limited in dynamic/knowledge-intensive tasks; 34–36% EM on Natural Questions	Improved factual accuracy via external knowledge grounding; 44–45% EM on Natural Questions
Response latency	Low latency (single-model inference)	Higher latency (retrieval + generation steps)
Cost efficiency	Higher cost for large-scale or complex models	More cost-efficient with external retrieval support
System complexity	Simple single-model architecture	Multi-component system (retriever + generator + embeddings)
Transparency	Low interpretability (internal parameters only)	Higher transparency via source-linked outputs
Hallucination Risk	Higher risk of factual errors; higher false positive (224 in LLM baseline experiment)	Reduced via retrieval grounding but depends on retrieval quality; lower false positive (114 in MEGA-RAG evaluation)
Domain Adaptability	Limited without retraining	High adaptability via external knowledge updates

C. Performance Discussion

The comparison presented in Table I highlights key differences in how Standalone LLM systems and RAG-based systems access and utilize knowledge during response generation. The following discussion interprets these differences by examining how each system behaves under different operational conditions.

In terms of accuracy and factual reliability, standalone LLM systems generally perform well in general-purpose tasks but may be limited in knowledge-intensive or rapidly changing domains due to their reliance on static training data [4]. In contrast, RAG-based systems address this limitation by incorporating external knowledge during inference. Empirical results from open-domain question answering show that RAG-based systems achieve around 44–45% Exact Match on the Natural Questions dataset, while standalone language models obtain roughly 34–36% under the same setting [8]. However, the effectiveness of RAG-based systems is dependent on retrieval quality, as irrelevant or noisy data may negatively affect output accuracy [1].

Regarding response latency, standalone LLM systems typically provide faster responses due to their single-model architecture [4]. Conversely, RAG-based systems introduce a retrieval phase that increases response time. However, architectural optimizations such as caching and parallel retrieval-generation can reduce latency, making performance dependent on system design [1].

From a cost efficiency perspective, standalone LLM systems may require higher computational resources and cost, particularly for large models [4]. RAG-based systems can achieve competitive performance using smaller models combined with retrieval mechanisms, making them more suitable in resource-constrained environments [3]. However, retrieval evaluation and optimization introduce additional computational overhead, and recent work has highlighted that end-to-end evaluation of RAG-based systems is expensive and inefficient [9].

In terms of system complexity, standalone LLM systems follow a relatively simple and unified architecture, making them easier to deploy and maintain [4]. RAG-based systems involve multiple components, including retrieval modules, embedding models, and external knowledge bases which increases flexibility but also introduces challenges in system integration, tuning, and maintenance [8].

For transparency and traceability, standalone LLM systems provide limited visibility into how responses are generated, as outputs are based on internal model parameters [4]. RAG-based systems improve transparency by linking responses to retrieved external sources, enabling better interpretability and verification [2]. This feature is particularly important in domains where evidence-based outputs are required.

Finally, in terms of domain adaptability and hallucination risk, standalone LLM systems are generally more suitable for broad applications but may struggle in specialised domains without retraining [4]. RAG-based systems demonstrate stronger adaptability by integrating domain-specific knowledge dynamically. Previous study shows that retrieval-based systems reduce hallucination-related errors, with false positives from 224 in standalone LLMs to 114 in retrieval-augmented systems [10]. However, this improvement is still dependent on retrieval quality as noisy or irrelevant retrieved information may still affect the reliability of output [8].

Overall, both systems demonstrate distinct trade-offs across performance dimensions. Standalone LLM systems prioritise speed and simplicity, while RAG-based systems emphasise accuracy, adaptability, and transparency. The relative performance of each approach depends on system configuration and application requirements rather than a universally superior design.

CONCLUSION

This study compared Standalone LLM systems and RAG-based systems in terms of architecture and key performance dimensions, including accuracy, latency, cost, system complexity, transparency, and adaptability. The analysis shows that standalone LLM systems are generally more efficient in terms of speed and simplicity due to their single-model architecture, while RAG systems provide improved factual reliability and adaptability by incorporating external knowledge during inference.

The key finding from this comparative analysis is that neither approach can be considered universally superior, as their effectiveness is highly dependent on the application context and system requirements. Standalone LLM systems are more suitable for general-purpose and latency-sensitive applications, whereas RAG systems are more appropriate for knowledge-intensive environments where updated and verifiable information is required.

This comparative study provides a structured reference that can support system designers in evaluating trade-offs between generative efficiency and knowledge accuracy when designing LLM-based applications. In practice, this enables more informed architectural decisions and reducing the risk of selecting unsuitable system designs and improving alignment between system capabilities and real-world application needs.

However, this study is limited by its conceptual nature and does not include experimental validation or quantitative benchmarking across real-world datasets. Future research may focus on empirical evaluation of both systems under controlled environments and explore hybrid architectures that further optimise the balance between retrieval efficiency and generative capability.

REFERENCES

- [1] Gao Y, Xiong Y, Gao X, Jia K, Pan J, Bi Y, Dai Y, Sun J, Wang M, Wang H. Retrieval-Augmented Generation for Large Language Models: A Survey [Internet]. arXiv.org. 2023. Available from: [https://arxiv.org/abs/2312.10997#:~:text=Retrieval%2DAugmented%20Generation%20\(RAG\)](https://arxiv.org/abs/2312.10997#:~:text=Retrieval%2DAugmented%20Generation%20(RAG))
- [2] Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, K ttler H, Lewis M, Yih W, Rock tschel T, Riedel S, Kiela D. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [Internet]. arXiv.org. 2021. Available from: <https://arxiv.org/abs/2005.11401>
- [3] Abu Laila D, Al-Na mneh Q, Aljawarneh M, Dhifallah W, Maabreh K. Retrieval-Augmented Generation With Large Language Models. Innovations and Ethical Dimensions of Large Language Models. 2025 Dec 19;73–98.
- [4] Ouyang L, Wu J, Jiang X, Almeida D, Wainwright CL, Mishkin P, Zhang C, Agarwal S, Slama K, Ray A, Schulman J, Hilton J, Kelton F, Miller L, Simens M, Askell A, Welinder P, Christiano P, Leike J, Lowe R. Training language models to follow instructions with human feedback. arXiv (Cornell University). 2022 Mar 4. Available from: <https://doi.org/10.48550/arxiv.2203.02155>
- [5] Brown T, Mann B, Ryder N, Subbiah M, Kaplan J, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A, Agarwal S, Herbert-Voss A, Krueger G, Henighan T, Child R, Ramesh A, Ziegler D, Wu J, Winter C, Hesse C, Chen M, Sigler E, Litwin M, Gray S, Chess B, Clark J, Berner C, McCandlish S, Radford A, Sutskever I, Amodei D. Language Models Are Few-Shot Learners. ArXiv (Cornell University) [Internet]. 2020 May 28;4. Available from: <https://arxiv.org/abs/2005.14165>
- [6] Microsoft, “Introducing the new Bing,” 2023. [Online]. Available from: <https://blogs.microsoft.com/blog/2023/02/07/introducing-the-new-bing/>
- [7] Perplexity AI, “How Perplexity Works,” 2024. [Online]. Available from: <https://www.perplexity.ai>
- [8] Zhao P, Zhang H, Yu Q, Wang Z, Geng Y, Fu F, Yang L, Zhang W, Jiang J, Cui B. Retrieval-Augmented Generation for AI-Generated Content: A Survey. Data Science and Engineering. 2026 Jan 2;11. Available from: <https://doi.org/10.1007/s41019-025-00335-5>
- [9] Salemi A, Zamani H. Evaluating Retrieval Quality in Retrieval-Augmented Generation. 2024 Jul 10. Available from: <https://doi.org/10.1145/3626772.3657957>.
- [10] Xu S, Yan Z, Dai C, Wu F. MEGA-RAG: a retrieval-augmented generation framework with multi-evidence guided answer refinement for mitigating hallucinations of LLMs in public health. Frontiers in Public Health. 2025 Oct 8;13.